Introduction

This is the final piece of coursework, which will count for 67% of the overall course mark. This coursework is composed of two parts, each with an equal weighting:

- Part A (straightforward): Tracking a square.
- Part B (challenging): You choose one from a list of options.

This assignment is open-ended in order to assess whether you (i) have a deep understanding of the material, (ii) have read beyond the lectures and (iii) are able to solve problems that you have not seen in previous assignments or that were not covered in lectures. You will also be assessed on your ability to clearly and rigorously define and solve mathematical problems. There is not a single, correct solution and trade-offs are inevitable. You will therefore have to demonstrate a critical understanding of the advantages, disadvantages and trade-offs of a solution.

Framework

You should use the MATLAB/SIMULINK files you have been given in previous assignments as a starting point. Please modify these as appropriate in order to complete the tasks below. Though you have already been provided with `pcode` that are answers of previous assignments, you are highly encouraged to use your own `m`-files.

You will be given the opportunity to implement your controller designed in Part A on the hardware during the hands-on session, which will be timetabled during the last two weeks of term. You are therefore recommended to implement and test your controller on the Simulink model given to you in Assignment 3 or 4. Because of time constraints and other unexpected issues, it may not be possible to test your controller designed in Part B during the hardware hands-on session. Note you should not include any results from the hardware session in the report.

Tasks

Part A: Tracking a square

In previous assignments you were asked to design a regulator that would take the system from an initial state to a different target state, while satisfying some constraints. You now have to modify the code to track a time-varying reference signal. In particular, you have to ensure that the mass of the *pendulum* can track a square in the XY-plane with a given amount of error. It is up to you to define the four corners of the square, the maximum error and the time it takes to complete the square, etc.

You should pay particular attention to the following:

- There is no point in implementing constrained MPC if the closed-loop response is the same as for an unconstrained linear controller. You have to compare your results against an unconstrained RHC law with the same cost function on the states and inputs as the constrained controller.

- You should compare results for both the linear and nonlinear simulation.
- Your sample time should be larger than the time it takes to solve a single QP.
- The inputs are constrained between -1 and 1. These are hard, physical constraints.
- The minimum and maximum values that the cart can take in the XY-plane are defined in the variables `xRange` and `yRange` in the file **Params_Simscape.mat**. These cannot be changed and represent physical limits, which cannot be violated. Please bear these values in mind and allow for some safety margin when defining constraints on the states.
- The starting point of the cart is defined by the variables `xZero` and `yZero`. You are allowed to change these, as long as they are within the above-mentioned range.

Part B: Choose one

You have to choose from one of the following options — please choose only one:

1. Discuss the practical challenges that arise with MPC in a continuous process industry plant, both (i) during the implementation phase and (ii) during the entire life cycle.
2. Implement a controller that ensures the pendulum tracks something other than a square, e.g. a triangle, circle and/or any other interesting path in the XY-plane.
3. Implement a tracking controller that uses a preview of a time-varying reference and demonstrate that this does better than a non-preview controller.
4. Implement a predictive controller with a non-quadratic cost on the states and inputs.
5. Implement a predictive controller that can guarantee zero tracking error in the presence of a constant disturbance and/or plant-model mismatch.
6. Implement a predictive controller that requires less computational resources, compared to the controllers designed in previous assignments, without changing the solution to the QPs.
7. Implement a predictive controller that requires less computational resources, compared to the controllers designed in previous assignments, without significantly changing the closed-loop behaviour.
8. Implement a predictive controller that uses 'blocking' for the input sequence.
9. Implement a predictive controller that has different control and prediction horizons.
10. Implement a predictive controller that uses the so-called delta-u formulation, i.e. the change in input $\Delta u_k := u_k - u_{k-1}$ is included in the cost and/or constraints.
11. Implement an explicit predictive control law. See the documentation for the MATLAB Model Predictive Control Toolbox or the MPT Toolbox (`http://people.ee.ethz.ch/~mpt/3/`), as well as the paper `http://dx.doi.org/10.1016/S0005-1098(01)00174-1`
12. Implement a nonlinear predictive controller, e.g. using ACADO (`https://acado.github.io`), ICLOCS (`http://www.ee.ic.ac.uk/ICLOCS`), YALMIP (`https://yalmip.github.io`) or any other suitable software package that you can find.

13. Implement a predictive controller with a guarantee of closed-loop stability and/or invariance. You might find the MPT Toolbox (`http://people.ee.ethz.ch/~mpt/3/`) useful for computing an invariant set.
14. Implement a predictive controller that minimizes the time to reach a target state.
15. Implement an economic predictive controller and compare the performance to one based on a standard tracking/regulator formulation, as implemented in previous assignments.
16. Implement a robust predictive controller that guarantees constraint satisfaction and/or a certain amount of closed-loop tracking error despite the presence of uncertainties.
17. Implement any predictive control scheme that you have come across and have found interesting, but is not in the list above.

As with Part A, there is no point in implementing a predictive controller if the same response can be achieved with an unconstrained linear controller. You therefore have to demonstrate the advantage of your method compared to a similar, unconstrained controller, if appropriate and possible.

For all the options above, marks will be given for elegant discussions that demonstrate a deep understanding, compared to trivial solutions that demonstrate a superficial understanding. Note that some of the options above might wrongly be considered to be 'easier' than others.

Submission details

Your should submit your report, together with all the MATLAB/SIMULINK files needed to generate the results, as a single ZIP file on Blackboard. Do not include any external software packages, if used.

Report

Your report should be submitted as a PDF and adhere to the following:

- Be written with the IEEE Control Systems Society conference paper A4 template.

  A template for LaTeX can be downloaded from
  `http://css.paperplaza.net/conferences/support/tex.php`
  A template for Word can be downloaded from
  `http://css.paperplaza.net/conferences/support/word.php`
- Be in two-column format.
- The title should be your full name and CID.
- Part A and Part B together should not be longer than 6 columns (3 pages), subject to the following additional requirements:
  - Part A should not be longer than four columns (2 pages).
  - Part B should not be longer than four columns (2 pages).

  In other words, one of the parts cannot be longer than three columns (1.5 pages). It is up to you whether you want to assign an equal amount of space to both parts.

Please make sure you clearly define the optimal control problem as well as the optimization problem and list the numerical values that parameterize the controller, e.g. the sample period, horizon length, cost weights, constraints, etc. You do not need to define symbols that have already been defined in previous assignments. However, all new symbols have to be clearly defined. A reader should ideally be able to replicate your results without needing to refer to your code.

Code

You should also write and include the following two MATLAB scripts in the ZIP file:

PartA.m Runs all MATLAB/SIMULINK simulations to generate the data and figures included in Part A of the report.

PartB.m Runs all MATLAB/SIMULINK simulations to generate the data and figures included in Part B of the report.

Marking allocation

- Clarity of expression, grammar, legibility of figures, typesetting and presentation. [25%]
- Are the optimal control problems and algorithms clearly defined, if necessary, with no repeat of material already used in previous assignments and lectures? Are all new variables defined? Has the literature been properly referenced? [25%]
- Are the mathematics correct and the numerical results replicable? Are the numerical values for all control parameters given? Have trade-offs, advantages and disadvantages been explored and discussed? [20%]
- Does the report discuss concepts beyond that covered in lectures? Does the discussion demonstrate a deep understanding of the presented material? [30%]

Nikolaos Altiparmakis
01267575

***Abstract*—We will implement a predictive control scheme to track a specific square using the gantry crane's pendulum mass. Several parameters, including the sampling time and the horizon length will be varied and their impact will be discussed. Later, we will derive and implement a predictive control scheme with non-quadratic costs for the states and the inputs. The controller will be implemented on the grantry crane and results will be discussed.**

I. Square Tracking

A. Target Calculator

When implementing control schemes in the gantry crane, we had a single point as target. Now that we must track a square, we could split the problem into four partial problems in each of which we will have a corner of the square as target. A target calculator will replace the constant target input in the controller. T is now a constant target and consists of four points, which define the square. It is up to the target calculator function to choose which one of them to set as a target for the predictive control problem.

Algorithm 1 Target Calculator

Input: T, x, i
Output: $target$, i
  $s \leftarrow (T(i,1), T(i,2))$
  $e \leftarrow 10^{-3}$
  $\alpha \leftarrow |(T(1,1), T(1,2)) - (T(2,1), T(2,2))|$
  $c \leftarrow (x(1), x(3))$
  if $|x(2)| \leq e$ **and** $|x(4)| \leq e$ **and** $|s - c| \leq \frac{1}{5}\alpha$ **and** $i \neq 4$ **then**
    $i \leftarrow i + 1$;
  end if
  $target \leftarrow (T(i,1), 0, T(i,2), 0, 0, 0, 0, 0)$
  return target, i

B. Constraints and Parameters

We will choose to track the square defined by the points $(a,b), (b,b), (b,a), (a,a)$ where $a = 0.208$ and $b = 0.416$. We will consider hard constraints both on the inputs and on the position of the crane. In this case, the hard constraint on the position will not be violated, but it is included to address the general case of tracking a square, in which the square might be near the borders. We will try to limit the angle by choosing an appropriate Q. We will choose $R = 0.01I_2$, $Q = diag(Q_p, 0.01, Q_p, 0.01, Q_\theta, 0.01, Q_\theta, 0.01)$ and $P = Q$. Our task is for the pendulum mass to track the square, which means that the angles of the pendulum should not get very large.

TABLE I: Parameters used to test the controller. T_{sim} is the time needed to complete the square, $\hat{e}_{\theta,\psi}$ is the maximum angle deviation, $\bar{t}_{opt}$ is the mean optimisation time.

	$T_s[s]$	N	Q_p	Q_θ	$T_{sim}[s]$	$\hat{e}_{\theta,\psi}[deg]$	$\bar{t}_{opt}[ms]$
1	0.1	15	5	12	7.5	4.25	0.65
2	0.2	15	5	12	8.4	4.53	0.68
3	0.1	10	5	12	7.8	4.52	0.61
4	0.1	15	5	20	8.4	3.6	0.69
5	0.1	15	10	12	7.4	5.83	0.71

C. Discussion

We have conducted a series of experiments by varying certain parameters as in Table I. The maximum angle error is approximated as $\hat{e}_{\theta,\psi} = max(max(\theta(kT_s)), max(\psi(kT_s))), k \in \mathbb{N}$.

The optimisation time at each step depends on the number of constraints and the horizon length. We will need to make sure that the choices of the constraints with the horizon length are such, that an optimisation problem can be solved in time T_s.

The time it takes to track the square depends on the overshoots and Q_θ. When Q_θ is large, we notice the crane waits for the pendulum to relax before starting to move towards the next target points. On the contrary, if we have a large T_s, then there will be an overshoot, which will also increase the tracking time, since extra time is needed in order for the crane to stabilise into a point and move to the other. According to the sampling time, we will need to have an appropriate choice of parameters to avoid an overshoot and waiting before moving to another target point.

When implementing a control scheme on hardware, there is a minimum combination of sampling time and horizon length, beyond which the system will not be able to solve the optimisation problems in time. Now that we have added a target calculator for each, more time will be needed to solve the optimisation problems,

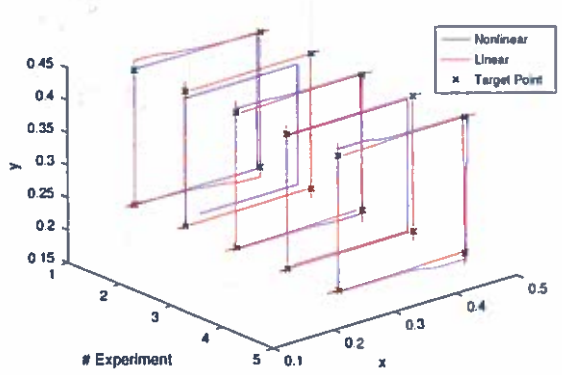


Fig. 1: Curves on the plane by the gantry crane for the parameters as given in Table I.

therefore we will need to take that into account and increase T_s/decrease N accordingly, if needed.

The target calculator will need to be executed at each time step, therefore increasing the runtime of the control algorithm. More sophisticated algorithms could be implemented. For example, we could split the matrix $L = \begin{bmatrix} L_1 \\ 0 \end{bmatrix} + \begin{bmatrix} 0 \\ L_2 \end{bmatrix}$ and instead of Lx_e, use $\begin{bmatrix} L_1 x_e \\ L_2 x_c^* \end{bmatrix}$, where x_c^* is the next target point.In such cases, the crane will do a continuous trajectory instead of a piecewise continuous and the tracking time might decrease significantly. These 2 matrices will need to change at each time step, therefore more computations will take place. This is a trade-off of such methods: we might have a better tracking due to the algorithm, but we might need to use a smaller N or T_s, which, in turn, will have a negative impact on the controller's performance.

The controller is robust, regarding the behaviour in the non-linear model, when T_s is large enough. This might be explained by the fact that, for large T_s, the control signal generated is active for more time, therefore the system is affected much more by the system distrurbancies.

II. On non-quadratic cost optimisation

A. Formulation as linear program

Consider $J = \|P(x_N - x_e)\|_1 + \sum_{k=0}^{N-1} \|Q(x_k - x_e)\|_1 + \|R(u_k - u_e)\|_\infty$ as the cost function and the typical constraints. Many applications use ∞-norms and 1-norm cost functions which are transformed into linear costs using slack variables [1] [2]. We will do the same here and rewrite the program as:

$$\min_{w,u,s,t} 1^T s_N + \sum_{k=0}^{N-1} (1^T s_k + t_k) \quad (1)$$

$$w_{k+1} = Aw_k + Bv_k \quad (2)$$

$$\check{u} - u_e \le v_k \le \hat{u} - u_e \quad (3)$$

$$\check{c} - Dx_e \le Dw_k \le \hat{c} - Dx_e \quad (4)$$

$$-s_N \le Pw_N \le s_N \quad (5)$$

$$-s_k \le Qw_k \le s_k \quad (6)$$

$$-t_k \le Rv_k \le t_k \quad (7)$$

We may now construct the stage and terminal constraints as follows:

$$\underbrace{\begin{bmatrix} DA \\ -DA \\ 0 \\ 0 \\ Q \\ -Q \\ 0 \\ 0 \end{bmatrix}}_{\tilde{D}_Q^*} w_k + \underbrace{\begin{bmatrix} DB \\ -DB \\ I_m \\ -I_m \\ 0 \\ 0 \\ R \\ -R \end{bmatrix}}_{\tilde{E}_R^*} u_k + \underbrace{\begin{bmatrix} 0 \\ 0 \\ 0 \\ 0 \\ -I_n \\ -I_n \\ 0 \\ 0 \end{bmatrix}}_{\tilde{W}^*} s_k + \underbrace{\begin{bmatrix} 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ -1_m \\ -1_m \end{bmatrix}}_{\tilde{T}^*} t_k$$

$$\le \underbrace{\begin{bmatrix} \tilde{b} \\ 0 \end{bmatrix}}_{\tilde{b}^*} + \underbrace{\begin{bmatrix} \tilde{D} \\ 0 \end{bmatrix}}_{\tilde{D}_0^*} x_e + \underbrace{\begin{bmatrix} \tilde{E} \\ 0 \end{bmatrix}}_{\tilde{E}_0^*} u_e, k \in \{0, ..., N-1\} \quad (8)$$

and

$$\underbrace{\begin{bmatrix} P \\ -P \end{bmatrix}}_{\tilde{D}_N} w_N + \underbrace{\begin{bmatrix} -I_n \\ -I_n \end{bmatrix}}_{\tilde{W}_N} s_N \le 0 \quad (9)$$

We can then construct the trajectory constraint matrices:

$$\bar{D}_Q = \begin{bmatrix} I_N \otimes \tilde{D}_Q^* & 0 \\ 0 & \tilde{D}_N \end{bmatrix}, \bar{E}_R = \begin{bmatrix} I_N \otimes \tilde{E}_R^* \\ 0 \end{bmatrix}$$

$$\bar{W} = \begin{bmatrix} I_N \otimes \tilde{W} & 0 \\ 0 & \tilde{W}_N \end{bmatrix}, \bar{T} = \begin{bmatrix} I_N \otimes \tilde{T} \\ 0 \end{bmatrix}$$

$$\bar{b} = \begin{bmatrix} 1_N \otimes \tilde{b}^* \\ 0 \end{bmatrix}, \bar{D}_0 = \begin{bmatrix} I_N \otimes \tilde{D}_0^* \\ 0 \end{bmatrix}, \bar{E}_0 = \begin{bmatrix} I_N \otimes \tilde{E}_0^* \\ 0 \end{bmatrix} \quad (10)$$

If the equation $\begin{bmatrix} w_0 \\ \bar{w} \end{bmatrix} = \underbrace{\begin{bmatrix} I \\ \Phi \end{bmatrix}}_{\tilde{\Phi}^*} w_0 + \underbrace{\begin{bmatrix} 0 \\ \Gamma \end{bmatrix}}_{\tilde{\Gamma}^*} \bar{u}$ is used, then the original problem can be transformed into the following LP:

$$\min_{\bar{u},\bar{s},\bar{t}} \begin{bmatrix} 0 & 1 & 1 \end{bmatrix} \begin{bmatrix} \bar{u} \\ \bar{s} \\ \bar{t} \end{bmatrix} \quad (11)$$

$$\begin{bmatrix} \bar{D}_Q \tilde{\Gamma}^* + \bar{E}_R & \bar{W} & \bar{T} \end{bmatrix} \begin{bmatrix} \bar{u} \\ \bar{s} \\ \bar{t} \end{bmatrix} \le \lambda$$

where $\lambda = \bar{b} - \bar{D}_Q\tilde{\Phi}^* x_0 - [\bar{D}_Q\tilde{\Phi}^* + \bar{D}_0 \cdot (1_N \otimes I_N)]x_e - \bar{E}_0(1_N \otimes I_N)u_e$, $\bar{s} = \begin{bmatrix} s_0^T & \dots & s_N^T \end{bmatrix}^T$ and $\bar{t} = \begin{bmatrix} t_0^T & \dots & t_{N-1}^T \end{bmatrix}^T$

We have not taken into account any constraints for the slack variables. All slack variables should be non-negative. However, this is guaranteed by the constraints given in the linear program already, so they do not need to be taken into account.

B. Simulations

We will now run some experiments on the gantry crane. We will use the same type of Q, $P = Q$ and $R = 45I_3$. We will also use the same target calculator.

TABLE II: Parameters used to test the controller. T_{sim} is the time needed to complete the square, $\hat{e}_{\theta,\psi}$ is the maximum angle deviation, $\bar{t}_{opt}$ is the mean optimisation time.

	$T_s[s]$	N	Q_p	Q_θ	$T_{sim}[s]$	$\hat{e}_{\theta,\psi}[deg]$	$\bar{t}_{opt}[ms]$
1	0.1	15	150	40	7	7.36	19.6
2	0.1	15	200	110	8	6.9	21.7
3	0.1	10	300	160	8.6	5.91	16.9
4	0.3	15	400	250	9.6	5.64	20.1
5	0.2	15	400	150	5.8	5.49	20.5

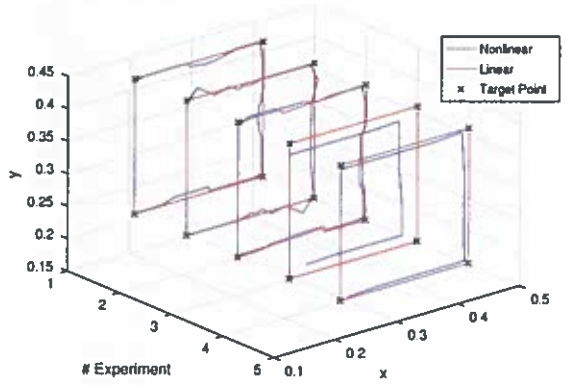


Fig. 2: Curves on the plane by the gantry crane for the parameters as given in Table II.

We did not observe any improvements regarding this cost function over the quadratic costs used in the previous analysis. It takes much more time to complete a single optimisation problem, there is a larger angle deviation and the tracking is completed in roughly the same time (Table II).

C. Discussion

Although we have constructed a linear program instead of a quadratic, as we had in the standard case of quadratic costs, there are now more decision variables, due to the introduction of the slack variables needed to convert the program to an LP. Therefore, solving the program will require additional computational resources. It should be noted, that the time it takes to solve a program is implementation dependent: MATLAB's *linprog* (which will be used in this case) is known to face problems in some situations [3].

It was observed that the individual costs compete with each other in the following sense: for large Q_p, there is a dead beat control, while for large Q_θ, the controller is idle. The notion of competing costs is discussed on [4]. Therefore, a suitable choice of Q_p and Q_θ is needed to ensure convergence.

This controller has a more aggressive control action. The squares are tracked perfectly in the linear case and this increases the sensitivity of the controller to disturbances, as seen in the nonlinear case (Fig. 2). More specifically, we noticed the gantry crane going back and forth with high velocity in order to make the angle small, but there was no helpful outcome as a result of that action.

With the increase of time it takes to solve an optimisation problem, it might not be possible to implement this controller into the hardware. In order to fix this issue, T_s must be larger or N must be smaller than in the quadratic case. Also, sophisticated LP solvers like NAG [5] or LP solvers which are optimised to address 1-norm and ∞-norm costs [6] could have a positive impact. Otherwise, offline optimisation should be considered as an alternative. Considering x as a vector of parameters, explicit state feedback laws can be constructed by the so-called multiparametric linear programs [7].

References

[1] H. Fu, M. Ng, M. Nikolova and J. Barlow, "Efficient Minimization Methods of Mixed l2-l1 and l1-l1 Norms for Image Restoration", SIAM Journal on Scientific Computing, vol. 27, no. 6, pp. 1881-1902, 2006.

[2] Y. Zhang, J. Yin and B. Cai, "Infinity-norm acceleration minimization of robotic redundant manipulators using the LVI-based primal–dual neural network", Robotics and Computer-Integrated Manufacturing, vol. 25, no. 2, pp. 358-365, 2009.

[3] "MPT - The Multi-Parametric Toolbox for Matlab", People.ee.ethz.ch, 2017. [Online]. Available: http://people.ee.ethz.ch/~mpt/2/docs/faq.php#3.3. [Accessed: 19-Mar-2017].

[4] C. Rao and J. Rawlings, "Linear programming and model predictive control", Journal of Process Control, vol. 10, no. 2-3, pp. 283-289, 2000.

[5] "The NAG Toolbox for MATLAB", www.nag.co.uk, 2017. [Online]. Available: https://www.nag.com/nag-toolbox-matlab. [Accessed: 19 Mar-2017].

[6] Y. Gupta, "A Modified Simplex Method for Solving 1-Norm Minimization Problem in Model Predictive Control", The Canadian Journal of Chemical Engineering, vol. 85, no. 2, pp. 199-207, 2008.

[7] A. Bemporad, F. Borrelli and M. Morari, "Model predictive control based on linear programming - the explicit solution," in IEEE Transactions on Automatic Control, vol. 47, no. 12, pp. 1974-1985, Dec. 2002.

Model Predictive Control with Disturbance Rejection

Jun Yu, 01234152, *Department of Electrical and Electronic Engineering, Imperial College London*

***Abstract*— This report presents the application of MPC in order to control a gantry crane model to track certain targets, while an improvement on error-reducing is also implemented.**

I. Introduction

This report is divided into two main sections. The first one is about implementing model predictive control to drive a pendulum to track a square and comparing the performances between different controllers and on different models. The second part is about studying the techniques of disturbance rejection and constructing an offset-free MPC model to obtain better performance.

II. Square Tracking

In this section, the performance of a constrained MPC controller and an unconstrained RHC controller is first presented and analyzed with a linear model. After that, the constrained MPC controller is compared between a linear and a nonlinear model.

A. Constrained & Unconstrained Linear MPC

In order to compare the performances between constrained and unconstrained controller equally, only hard constraints are considered in this subsection. Therefore, as slack variables are not included, both the controllers share an identical cost function, and they are tested in a linear model.

The constrained MPC design is formulated as,

$$\begin{aligned} \min_{u_0,\cdots,u_{N-1}} \ & \|x_N - x_e\|_P^2 + \sum_{k=0}^{N-1} \|x_k - x_e\|_Q^2 + \|u_k - u_e\|_R^2 \\ \text{subj. to} \ & x_{k+1} = Ax_k + Bu_k, \ \ k = 0, \cdots, N-1 \\ & \check{u} \le u_k \le \hat{u}, \ \ k = 0, \cdots, N-1 \\ & \check{c} \le Dx_k \le \hat{c}, \ \ k = 1, \cdots, N \end{aligned} \tag{1}$$

where r is the reference signal and the target point (x_e, u_e) is given by,

$$\begin{bmatrix} A - I & B \\ C & 0 \end{bmatrix} \begin{bmatrix} x_e \\ u_e \end{bmatrix} = \begin{bmatrix} 0 \\ r \end{bmatrix} \tag{2}$$

And the unconstrained RHC is constructed as in form in (1) while the constraints on u_k and Dx_k are neglected during the computation. With the knowledge of system dynamics and cost function in (1), the off-line computation for unconstrained RHC of state feedback gain K can be completed.

As for the constraints listed in (1), in order to implement a more explicit constraint on the pendulum position, matrix D is

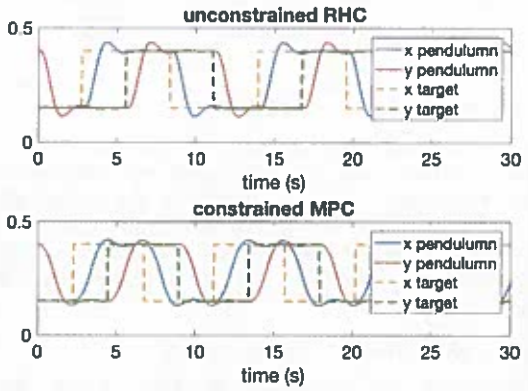


Figure 1. Position Tracking

modified into the following forms,

$$D = \begin{bmatrix} 1 & 0 & 0 & 0 & r & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & r & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 \end{bmatrix} \tag{3}$$

while r is the length of pendulum.

By implementing this adjustment, the first two elements of Dx_k becomes $(x_1 + rx_5)$ and $(x_3 + rx_7)$, which are the approximation of $(x_1 + r\sin x_5)$ and $(x_3 + r\sin x_7)$, assuming swing angle x_5 and x_7 are small enough. Therefore, the state constraints are now directly applied to the bob itself, instead of the moving support, as in the default setting.

Moreover, state constraint $(\check{c}, \hat{c})$ on the position is designed to vary along with the target points and four sets of constraints are applied, while all of them allow a 0.01 position offset along the paths. And the angle constraint is tightened down to 1.5° in order to reduce the swinging.

By defining a certain tolerance on the crane position and velocity on both axis, four target points are switched in order. Then the moving support is designed to track a certain square as the response displayed in Fig. 1. All the numerical settings of parameters are placed in the appendix.

Meanwhile, since no constraint is added to the input for the unconstrained RHC controller, the violation on the input physical constraint is inevitable. Therefore, a saturation statement is added to the input, such that the control input cannot exceed the physical constraints at each iteration and the simulation becomes more practical.

As it shown, by adding constraints into the computation, the pendulum is able to complete a better square tracking as its

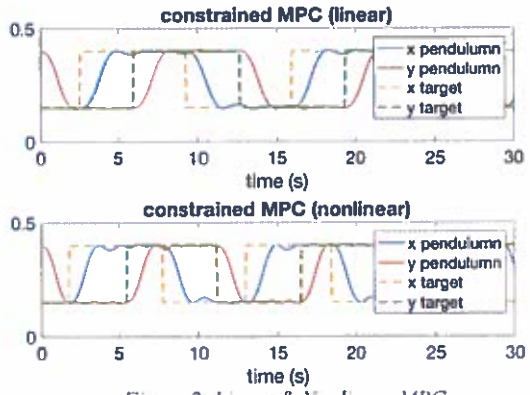


Figure 2. Linear & Nonlinear MPC

position overshoot and angle swinging are comparatively smaller. However, it is also noticed that, the unconstrained RHC controller, which does not need any on-line computation, has a much quicker response. Its average optimization time is only 7.73×10^{-6}s while this figure of the constrained MPC solver is 0.009s.

B. Linear & Nonlinear Constrained MPC

Considering soft constraints, the MPC design can be rewritten as,

$$\min_{u_0,\cdots,u_{N-1}} \|x_N - x_e\|_P^2 + \sum_{k=0}^{N-1} \|x_k - x_e\|_Q^2 + \|u_k - u_e\|_R^2$$

$$\text{subj. to} \quad x_{k+1} = Ax_k + Bu_k, \ k = 0,\cdots,N-1 \qquad (4)$$

$$\check{u} \le u_k \le \hat{u}, \ k = 0,\cdots,N-1$$

$$\check{c} - s_k \le Dx_k \le \hat{c} + s_k, \ k = 1,\cdots,N$$

while the target point (x_e, u_e) is also given by condition (2).

The parameter settings are also listed in the appendix while the simulation results of both models are demonstrated in Fig. 2. By adding soft constraints, the position offset is able to be tightened to 0.005, while the cost function is also modified to apply less weight on the angle because the constraints are softened and the controller is more feasible. Consequently, the response is more smooth since the oscillation is reduced. However, a longer computation time is required in this case while the average computation time is up to 0.0174s and hence the sampling time is also adjusted in (4).

In comparison to the nonlinear model, the linear MPC is able to complete a more accurate tracking since all the computation is based on the linearization of the nonlinear model in the first place. Hence when simulating the nonlinear plant, model mismatches is inevitable. Meanwhile, the model mismatches also allow the states of nonlinear model drop faster down to the tolerance and therefore it has a quicker turning at the corners than the linear model does.

III. Disturbance Rejection

As mentioned above, because of linearization, model mismatch exists in the nonlinear model and system performance is disrupted consequently. Therefore, disturbance rejection is introduced here in order to compensate the error and construct an offset-free model. The system performance is tested on square tracking.

A. Computation of Augmented Disturbance Model

In order to implement disturbance rejection, an augmented disturbance model is constructed as,

$$\begin{cases} x_{k+1} = Ax_k + Bu_k + B_d\hat{d} \\ y_k = Cx_k + C_d\hat{d} \end{cases}, \ k = 0,\cdots,N-1 \qquad (5)$$

Disturbance is considered as a constant in the dynamic prediction while matrices B_d and C_d are selected to describe the effect of the disturbance. The dimension of disturbance $\hat{d}$ is considered to be equal to the number of the states.

Hence the MPC problem is reformulated as follows,

$$\min_{u_0,\cdots,u_{N-1}} \|x_N - x_e\|_P^2 + \sum_{k=0}^{N-1} \|x_k - x_e\|_Q^2 + \|u_k - u_e\|_R^2$$

$$\text{subj. to} \quad x_{k+1} = Ax_k + Bu_k + B_d\hat{d}, \ k = 0,\cdots,N-1$$

$$\check{u} \le u_k \le \hat{u}, \ k = 0,\cdots,N-1 \qquad (6)$$

$$\check{c} \le Dx_k \le \hat{c}, \ k = 1,\cdots,N$$

$$x_0 = \hat{x}(t)$$

And the target point (x_e, u_e) is now given as follows,

$$\begin{bmatrix} A-I & B \\ C & 0 \end{bmatrix}\begin{bmatrix} x_e \\ u_e \end{bmatrix} = \begin{bmatrix} -B_d\hat{d} \\ r - C_d\hat{d} \end{bmatrix} \qquad (7)$$

In (2), target point (x_e, u_e) is identical to the reference signal $(r, 0)$, while in this section, this condition no longer stands. In (7), the target point is adjusted by both the reference signal and the estimated disturbance.

Note that there is a different notation between x_k and $x(t)$, as x_k represents the predictive state that starts from time instant t and $x(t)$ stands for the current value, as it is suggested in [1].

Meanwhile, disturbance and initial state are estimated by a state observer for the MPC regulator, which is given by,

$$\begin{bmatrix} \hat{x}(t+1) \\ \hat{d}(t+1) \end{bmatrix} = \begin{bmatrix} A & B_d \\ 0 & I \end{bmatrix}\begin{bmatrix} \hat{x}(t) \\ \hat{d}(t) \end{bmatrix} + \begin{bmatrix} B \\ 0 \end{bmatrix}u(t)$$

$$+L\left(-y_k + \begin{bmatrix} C & C_d \end{bmatrix}\begin{bmatrix} \hat{x}(t) \\ \hat{d}(t) \end{bmatrix}\right) \qquad (8)$$

Comparing with the MPC model in (1), an identical cost function is shared while the inequality constraint needs to be modified. All the notations are the same as in [2].

First, the stage constraint is rewritten as,

$$\bar{D}x_k + \bar{E}u_k + \bar{D}_d\hat{d} \le \bar{b} \qquad (9a)$$

with an extra matrix $\bar{D}_d$ defined as,

$$\bar{D}_d = \begin{bmatrix} DB_d \\ -DB_d \\ O_{2mxn} \end{bmatrix} \qquad (9b)$$

After that, the trajectory constraint and the new matrix $\bar{D}_d$ are expressed as follows,

$$\bar{D}\bar{x} + \bar{E}\bar{u} + \bar{D}_d\hat{d} \le \bar{b} \qquad (10a)$$

$$\bar{D}_d = \begin{bmatrix} \bar{D}_d \\ \vdots \\ \bar{D}_d \end{bmatrix} \tag{10b}$$

In the end, based on (6) and (10a), the inequality constraint is reformulated as,

$$F\bar{u} \le \bar{b} + Jx_0 + Lx_e + Tu_e + J_d\hat{d} \tag{11a}$$

$$J_d = -\bar{D}_d \tag{11b}$$

By having inequality constraint (11a), MPC solver is now able to compute the solution to (6).

B. Important Conditions

(i) Based on the statements in [3], in order to guarantee observability on the state and disturbance, (C, A) is detectable and matrix $\begin{bmatrix} A-I & B_d \\ C & C_d \end{bmatrix}$ must have full column rank.

(ii) As for the observer, consider gain $L = \begin{bmatrix} L_x \\ L_d \end{bmatrix}$ consists of two separate gain corresponding to the state and the disturbance. To ensure stability of the observer, rank of L_d must be equal to the number of disturbance.

(iii) Assume condition (i) and (ii) are satisfied, the steady state of the observer is given by (7).

In the following simulations, matrix B_d is considered as an identity matrix while matrix C_d is a zero matrix since in the Simulink crane model, output signals are identical to the states. Condition (i) is satisfied by this setting while condition (ii) is fulfilled by implementing a Kalman filter in the model.

C. Simulation Results

Sharing same set of parameters, the disturbance rejection is tested on an unconstrained RHC controller and a constrained controller, as it shown in Fig. 3. The numerical values of the parameters are listed in the appendix.

It is first noticed from the simulation that both of the controllers can achieve offset-free tracking while there is no overshoot on the position and the oscillation on the angle is much smaller.

Similarly, the unconstrained RHC does not require any on-line computation and has a relatively fast response. However, without implementing any direct constraint on the states, unconstrained RHC causes comparatively larger swing angle than constrained MPC does as they share identical parameters, as it demonstrated in Fig. 3.

As for the previous model in (4), because of the system mismatch, the pendulum cannot reach a target point precisely without any tracking error. Therefore, in order to refine the performance of square tracking and reduce the position error down to certain tolerance, a much higher weight is needed for the position in comparison with other states. However, the undesired large swinging and oscillation on the velocity and angle are consequently obtained, because the controller penalizes more on the position states.

By compensating the mismatch, less cost weight and constraints are now needed for the position states while the pendulum can still reach the target point accurately. The cost

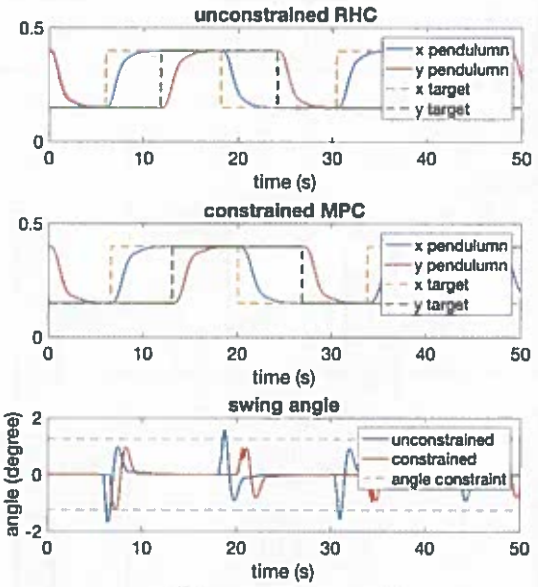


Figure 3. offset-free MPC

matrices are adjusted and the state constraints on the position are also loosen. Moreover, unlike the previous setting, the state constraints $(\check{c}, \hat{c})$ no longer varies along with the target points and the modification on matrix D in (3) is not needed in this case. As a consequence, more attention is paid by the MPC solver to stabilize the acceleration, swinging and oscillation on the states during the operation.

In summary, by implementing disturbance rejection, the performance of tracking is improved dramatically in both models and more importantly the pendulum is able to reach the target point smoothly and accurately.

IV. Conclusion

Model Predictive Control is successfully implemented in section II with two different controllers and two different models, while their performances are compared and analyzed. As in section III, disturbance rejection is studied. A linear Kalman filter is applied for the disturbance estimation and the system managed to achieve zero-error tracking. Future work includes implementing this technique on a model with different number of control states or disturbance, as the problem become more complicated in that case, while other techniques (e.g. blocking) can also be studied to reduce the amount of computation.

Appendix

Table I. Parameters Settings

	RHC & MPC (Hard)	Linear & Nonlinear (Soft)	Offset-free
T_s	$\frac{1}{25}$s	$\frac{1}{20}$s	$\frac{1}{25}$s
T_f	2s	1s	2s
Tolerance (Position)	5×10^{-4}	1×10^{-4}	5×10^{-7}

Tolerance (Angle)	5×10^{-5}	5×10^{-6}	N/A
Position Offset	0.01	0.005	N/A
Angle Constraint	1.5°	2°	1.25°
Q	$\mathrm{diag}(\begin{bmatrix}100 & 1 & 100 & 1 & 10 & 10 & 10 & 10\end{bmatrix}^T)$	$\mathrm{diag}(\begin{bmatrix}100 & 1 & 100 & 1 & 3 & 8 & 3 & 8\end{bmatrix}^T)$	$\mathrm{diag}(\begin{bmatrix}10 & 0.5 & 10 & 0.5 & 5 & 5 & 5 & 5\end{bmatrix}^T)$
R	$\mathrm{diag}(\begin{bmatrix}0.4 & 0.4\end{bmatrix}^T)$		$\mathrm{diag}(\begin{bmatrix}1 & 1\end{bmatrix}^T)$
P	$\mathrm{diag}(\begin{bmatrix}80 & 1 & 80 & 1 & 10 & 10 & 10 & 10\end{bmatrix}^T)$		$\mathrm{diag}(\begin{bmatrix}20 & 1 & 20 & 1 & 5 & 5 & 5 & 5\end{bmatrix}^T)$
ρ	N/A	750	N/A

ACKNOWLEDGMENT

I would like to express my appreciation to Dr. Eric Kerrigan for all his support and help during the whole lecture.

REFERENCES

[1] U. Maeder, F. Borrelli and M. Morari, "Linear offset-free Model Predictive Control", *Automatica*, vol. 45, no. 10, pp. 2214-2222, 2009.

[2] E. Kerrigan, "Predictive Control Lecture Note, Constrained LQR", 2017.

[3] G. Pannocchia and J. Rawlings, "Disturbance models for offset-free model-predictive control", *AIChE Journal*, vol. 49, no. 2, pp. 426-437, 2003.

Seunghyun Lee (00843738): Predictive Control Final Coursework

Seunghyun Lee

I. INTRODUCTION

Model Predictive Control (MPC) is an optimal control strategy based on numerical optimization. Future states of the system are predicted and optimized at a regular interval so that the control approach being taken is in its optimal state for a certain criteria (e.g. minimum energy required, minimum time etc.).

The system assigned for this module was a gantry crane system and from the first to fourth assignment in Predictive Control module, a Receding Horizon Controller (RHC) was designed as an Unconstrained Linear Quadratic Regulator (LQR) and developed to a Constrained LQR. Then tracking ability has been added by introducing target states.

Thus all together the following optimization problem was established:

$$\begin{aligned} \mathbf{t}^*(x_0, x_e) := arg\min_t \|x_N - x_e\|_P^2 + \sum_{k=0}^{N-1} (\|x_k - x_e\|_Q^2 \\ + \|u_k\|_R^2 + \|s_{k+1}\|_S^2 + \rho \|s_{k+1}\|_1) \end{aligned} \tag{1}$$

$$s.t. \; x_{k+1} = Ax_k + Bu_k, k = 0, 1, ..., N-1 \tag{2a}$$

$$\check{u} \leq u_k \leq \hat{u}, k = 0, 1, ..., N-1 \tag{2b}$$

$$\check{c} - s_k \leq Dx_k \leq \hat{c} - s_k, k = 1, 2, ..., N \tag{2c}$$

$$s_k \geq 0, k = 1, 2, ..., N \tag{2d}$$

Then, the final assignment encouraged variation in the cost function and/or constraint functions to vary the usage of this controller.

II. PART A: TRACKING A SQUARE

Part A requires a variation in target state. A single transition from an initial state to a different target state now had to be modified to track a time-varying reference signal. The time-varying reference signal has to ensure the pendulum can track a square in the XY-plane with a given amount of error.

A. Targeting module for time varying reference signal

The initial step to make the pendulum to track the square in XY-plane was to change the target in accordance to time through its connection to a clock.

Then the step taken to increase the accuracy and stability of pendulum tracking a square, was to add a tolerance value for the distance between the current position of the crane and the target position that has to be satisfied before the target is changed. This addition of criteria changed targeting module as the following:

1) Choose a tolerance value, and a maximum time that pendulum is given to get to the tolerance level in one target position (to not spend too much time at a single target)
2) If the norm of the difference of the current state vector and the target vector is below the tolerance level or the maximum time has been reached, update the target

This time varying reference signal to track a square was able to give structurally more stable tracking ability. However, moving onto the hardware implementation, the tracking was different from simulation. For more stabilized tracking, the constraints had to be tightened to give more accuracy to the target. However, the penalty matrices were loosened, since strong penalty matrices resulted in more penetration of the pendulum to keep the accurate XY position at every time step, which ultimately disturbed the tracking of the square.

As a result the parameter values used for tracking the square was the following:

$$Ts = 1/10; Tf = 1.5; N = ceil(Tf/Ts); T = 60$$

$$xTarget = \begin{bmatrix} 0.1 & 0 & 0.4 & 0 & 0 & 0 & 0 & 0 \\ 0.4 & 0 & 0.4 & 0 & 0 & 0 & 0 & 0 \\ 0.4 & 0 & 0.1 & 0 & 0 & 0 & 0 & 0 \\ 0.1 & 0 & 0.1 & 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

$$P = Q = \begin{bmatrix} 10 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 10 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 10 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 10 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

$$R = \begin{bmatrix} 0.01 & 0 \\ 0 & 0.01 \end{bmatrix}$$

B. Comparing tracking in Unconstrained and Constrained Predictive Controller

After the actual the time varying reference signal module was created the module was implemented in both unconstrained and constrained Receding Horizon Controller to see the effect of the constraints.

The main difference between the two controller that has been created was the existence of constraints. The existence of constraints in theory should result in less probability of

Running the simulation, it was able to show more unstable movement of the pendulum for the one implemented in unconstrained model. The Fig. 1 clearly shows more and greater ripples before getting to the steady state. Also, compared to the constrained MPC, there are greater magnitude

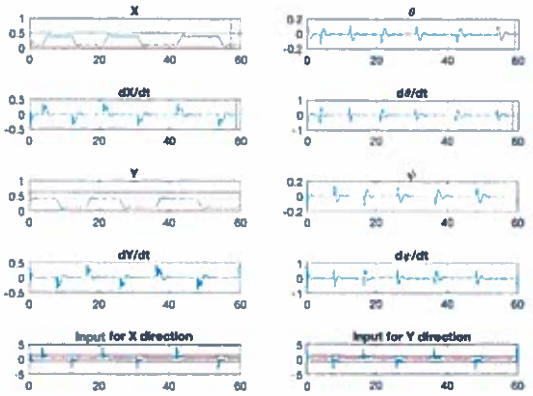


Fig. 1. Linear simulation of tracking square in unconstrained MPC

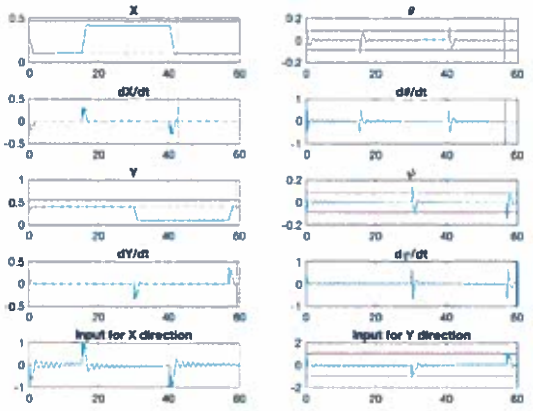


Fig. 2. Linear simulation of tracking square in constrained MPC

of inputs in x and y direction in order to keep the stability of the pendulum of the gantry crane.

However, it can also be clearly seen by comparing Fig. 1 and Fig. 2, that unconstrained MPC allows the pendulum to track the square more frequently than constrained MPC. This is because not only the constraints makes more computation to be done, but it also makes the controller to give more careful and non-aggressive outputs to ensure non-violation of the constraints.

III. PART B: IMPLEMENTING A PREDICTIVE CONTROLLER THAT USES DELTA-U FORMULATION

Delta-u formulation is another form of MPC, of which the change in input uk := uk uk1 is included in the cost and/or constraints. The Delta-u formulation of predictive controller forms the following optimization problem.

$$\min \sum_{k=1}^{N} \frac{1}{2}(\|z_k - r_k\|^2_{Q_z} + \frac{1}{2}\sum_{k=0}^{N-1} \|\Delta u_k\|^2_S) \tag{3}$$

$$s.t. x_{k+1} = Ax_k + Bu_k, k = 0,1,...,N-1 \tag{4a}$$
$$z_k = Cx, k = 0,1,...,N-1 \tag{4b}$$
$$\check{u} \le u_k \le \hat{u}, k = 0,1,...,N-1 \tag{4c}$$
$$\Delta\check{u} \le \Delta u_k \le \Delta\hat{u}, k = 0,1,...,N-1 \tag{4d}$$
$$\check{z} \le z_k \le \hat{z}, k = 1,2,...,N \tag{4e}$$

where z is the actual output, r is the setpoint, u is input, Δu is the change in input, Q_z and S are the weight for distance between current point and target point, and change in input respectively.

This allows manipulating the rate of change of input. The larger the penalty S is the less rate of change of input there will be, meaning the controller will try to get the pendulum of the gantry crane closest to the target point as much as possible with a single input before it change to another input.

A. Formulation into Quadratic Programming problem

In order to solve this optimization problem, the problem has to be reformed into a Quadratic Programming (QP) problem that can be solved either with `mpcqpsolver` or `quadprog` that can be utilised in MATLAB.

The form of the QP problem these problem solvers take is (for quadprog that was used for this part of assignment),

$$\min \frac{1}{2}x^T Hx + g^T x \tag{5a}$$
$$s.t. Ax \le b \tag{5b}$$
$$A_{eq}x = b_{eq} \tag{5c}$$
$$lb \le x \le ub \tag{5d}$$

Therefore the optimization problem had to be transformed into the QP problem in a different way from the previous assignments.

Starting from the prediction matrices, since $z_k = Cx_k$ from equation (4b), and $x_{k+1} = Ax_k + Bu_k$ from equation (4a),

$$z_k = CA^k x_0 + \sum_{j=0}^{k-1} CA^{k-1-j}Bu_j \tag{6}$$

Thus, for series of output values,

$$\begin{bmatrix} z_1 \\ z_2 \\ z_3 \vdots z_N \end{bmatrix} = \begin{bmatrix} CA \\ CA^2 \\ CA^3 \vdots CA^N \end{bmatrix} x_0 + \begin{bmatrix} CAB & 0 & \dots & 0 \\ CA^2B & CAB & \dots & 0 \\ CA^3B & CA^2B & \dots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ CA^NB & CA^{N-1}B & \dots & CAB \end{bmatrix}$$

Forming $Z = \Phi x_0 + \Gamma U$

R vector has been also created to be placed with the Z vector.

$$R = \begin{bmatrix} r_1 \\ r_2 \\ \vdots \\ r_N \end{bmatrix}$$

Then the weight matrix Q_z has been also modified as a sparse matrix form through $Q = I_N \otimes Q_z$ for direct computation with Z and R vectors, where $\otimes$ is the Kronecker product, and

I_N is the identity matrix of size N Thus, the cost function of $\frac{1}{2}(||z_k - r_k||^2_{Q_z}$ has been changed to $\frac{1}{2}(Z-R)^T Q(Z-R)$, which then can be computed down to $\frac{1}{2}U^T H_z U + g_z^T U$, where $H_z = \Gamma^T Q \Gamma, g_z = -\Gamma^T Q \Phi x_0 - \Gamma^T Q R$

Then with the similar method the sparse matrix form for Δu has been formed and added to the matrix. The constraint matrices has been also manipulated by expanding them to the size of multiples of horizon length N then all the constraints were stacked all together into one large function to be all placed into A and B in equation 5b.

IV. FURTHER DEVELOPMENT: OBSERVER FOR PREVIEW

The limitation encountered during the hardware implementation can also be regarded as disturbance, since the hardware model disrupts the index value that calls the required target.

In order to give more tolerance to this disturbance, it may be better to construct an observer, which echoes the action of the hardware model, to preview the motion that pendulum may take. This preview will give a resourceful feedback that would help the controller to give counter-balancing motion for more stabilized traverse of the pendulum. One of the most common observers used is Kalman Filter.

V. CONCLUSION

Model Predictive Control describes an approach to control design, not a specific algorithm. Therefore the user can interpret their own approach according to their needs. This gives wider spectrum of areas where MPC can be utilized.

Another area that has to be looked into and needs development would be Nonlinear MPC, which involves nonlinear constraints and cost functions. Many of the cost functions and constraints that has to be managed is not in linear form. Also since linear problem is a special form of nonlinear problem, development of Nonlinear MPC through developed packages such as YALMIP or IPOPT would allow the gantry crane system to be optimized in more different ways (e.g. minimizing the amount of energy used to move to the target, tracking in oval shape).

REFERENCES

[1] Srensen Matias, Model Predictive Control for an artirificial pancreas, Technical University of Denmark, 2007.